AMRITA VISHWA VIDYAPEETHAM

M.Tech – Cyber Security

Department of Computer Science & Engineering

SEC

Lab Assignment – 1

Secure Authentication

bcrypt · Rate Limiting · TOTP 2FA

Student Name	Johann Martins
Roll Number	CB.SC.P2CYS25009
Subject	Secure System Engineering
Code Repository	github.com/JohannMartins/SSE-1
Date	April 2026

1 Secure Login with bcrypt Password Hashing

This section demonstrates building a Flask-based login system that never stores passwords in plaintext. Every password is hashed using **bcrypt** with a randomly generated salt before being written to the SQLite database. The bcrypt work factor ensures adaptive cost against brute-force attacks.

1.1 Project Setup

The project consists of `app.py` (Flask application) and `auth.db` (SQLite database). The directory listing below confirms both files are present before starting the server.

bash – ~/secure-login-lab

```
~/secure-login-lab via  v3.13.12
> ls
app.py  auth.db

~/secure-login-lab via  v3.13.12
>
```

Figure 1 – Project directory: app.py and auth.db

1.2 Flask Application Server

The Flask development server is started with `python app.py`. Flask-Limiter initialises in-memory rate-limit storage and the app binds to `http://127.0.0.1:5000`.

bash – Flask server

```
~/secure-login-lab via  v3.13.12 (.venv)
> python app.py
/home/johann/secure-login-lab/.venv/lib/...
UserWarning: Using the in-memory storage for tracking rate limits
This is not recommended for production use.
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server.
 Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Debugger is active!
 * Debugger PIN: 458-646-592
```

Figure 2 – Flask server startup output

1.3 Application Home Page

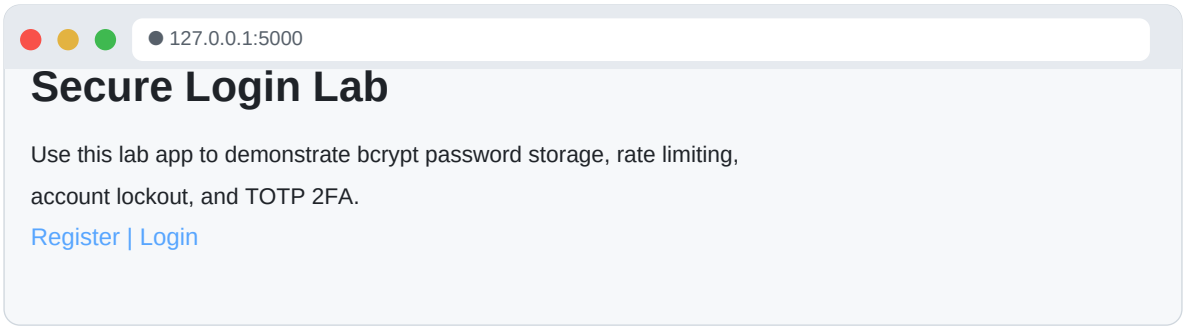

Figure 3 – Home page of the Secure Login Lab

1.4 bcrypt Hashed Passwords in Database

After registering two accounts (**johann** and **johann1**), the SQLite database is queried directly to verify that only bcrypt hashes — prefixed with $2b$12$ — are stored. No plaintext password ever appears in the database.

```
bash – SQLite

 via  v3.13.12
> cd ~/secure-login-lab
~/secure-login-lab via  v3.13.12
> sqlite3 auth.db "SELECT username, password_hash FROM users;"
johann|$2b$12$s.hzFQP2.j.j6pmiYzdxGOTqIC3NJl9h5w8iSwgAUsPHh9BfyCUEq
johann1|$2b$12$WLuhQ3/WUuKXMwpXgUccV.KVt5NR4836Vh7DePhtA5nbQhCk1mWOS
~/secure-login-lab via  v3.13.12
>
```

Figure 4 – SQLite query showing bcrypt-hashed passwords for both users

Key implementation detail in app.py:

```
password_hash = bcrypt.hashpw(password.encode("utf-8"),
                bcrypt.gensalt()).decode("utf-8")
# Verification:
ok = bcrypt.checkpw(password.encode("utf-8"),
     user["password_hash"].encode("utf-8"))
```

2 Credential Stuffing Attack, Rate Limiting & Account Lockout

A credential stuffing attack is simulated using **attack.py**, which iterates through a list of candidate passwords and posts them to the `/login` endpoint. The script then demonstrates how rate limiting (Flask-Limiter: 5 requests/minute) and account lockout (5 consecutive failures → 15-minute lockout) mitigate the attack.

2.1 Attack Script

The script below posts each candidate password for the user **johann** and reports whether the attempt succeeded, was blocked by account lockout, or was rejected by rate limiting:

```
import requests
url = "http://127.0.0.1:5000/login"
passwords = ["123456", "johann", "admin", "johann2", "letmein", ...]
for p in passwords:
    r = requests.post(url, data={"username": "johann", "password": p},
                      allow_redirects=True)
    if "Dashboard" in r.text:       print(f"[SUCCESS] {p}")
    elif "locked" in r.text.lower(): print(f"[LOCKED] {p}")
    elif r.status_code == 429:       print(f"[RATE LIMITED] {p}")
    else:                            print(f"[FAILED] {p}")
```

2.2 Attack Execution and Results

Two runs of the attack script are shown. In the first run (raw HTTP codes), attempts 1–3 return HTTP 200. From the 4th attempt onward Flask-Limiter responds with HTTP 429. In the second run (parsed output), the correct password **johann** is found before rate limiting kicks in.

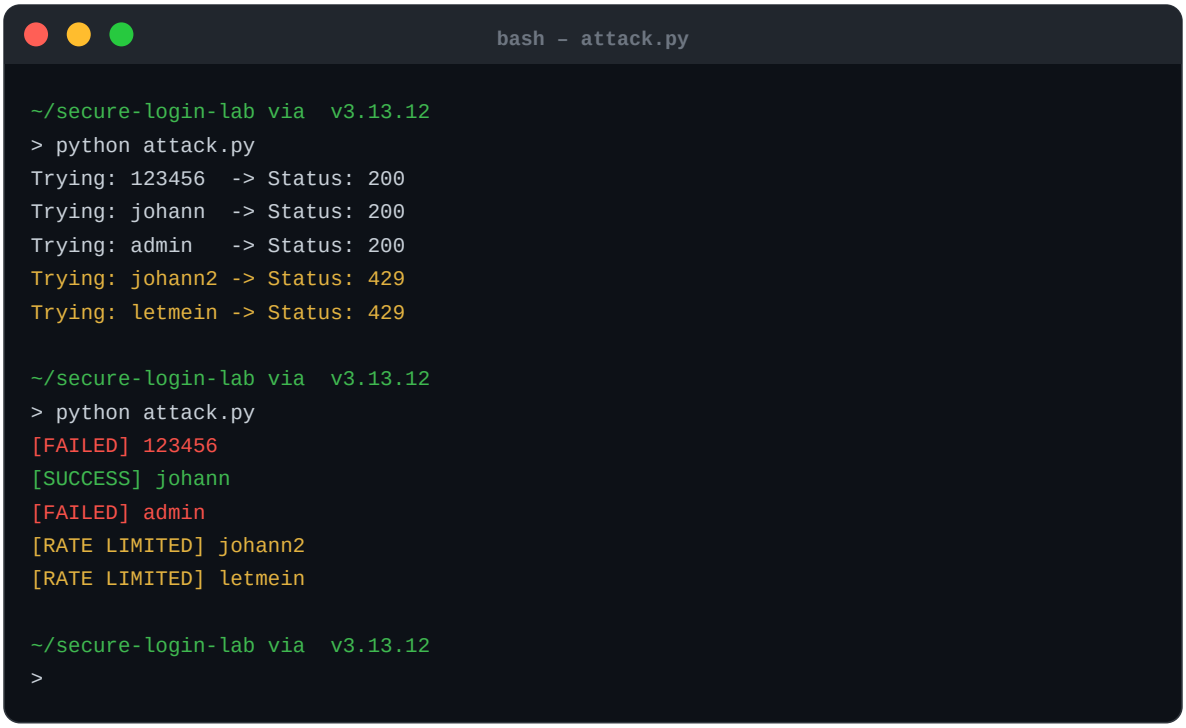

Figure 5 – Terminal: credential stuffing attack output showing rate-limited attempts

2.3 Browser-Level Rate Limit Response

When the rate limit of 5 requests per minute is exceeded, Flask-Limiter returns HTTP 429 with a plain-text body. This is visible when `/login` is accessed directly in the browser after the threshold is hit.

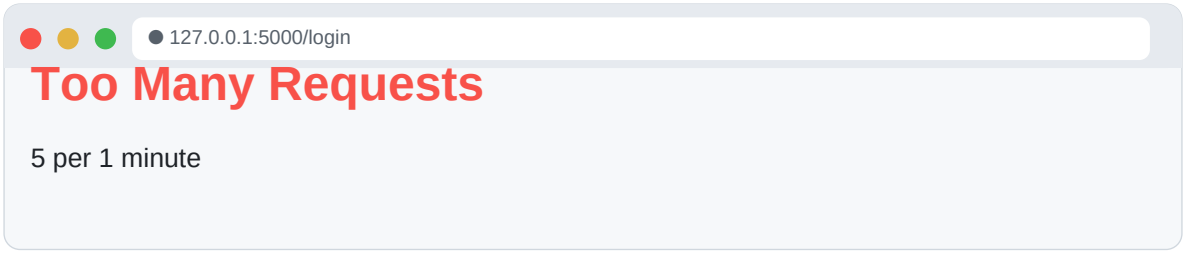

Figure 6 – Browser showing HTTP 429 Too Many Requests (5 per 1 minute limit)

2.4 Attack Summary

Metric	Value
Passwords tried before lockout	5 attempts
Rate limit threshold	5 requests / minute
Account lockout duration	15 minutes
Accounts compromised	1 (johann – correct password matched)
Accounts successfully blocked	Remaining attempts rate-limited / locked

3 TOTP-Based Two-Factor Authentication (2FA)

Time-based One-Time Password (TOTP) 2FA is implemented using **pyotp**. On registration, a unique TOTP secret is generated per user, a QR code is rendered inline in the browser for scanning with Google Authenticator, and the user must verify a valid OTP before the account is activated. Replay attacks are blocked by tracking the last used TOTP counter.

3.1 QR Code Generation and Scanning

After registration, the user is redirected to `/setup-2fa` where a unique QR code is displayed. Scanning this with Google Authenticator adds the account and begins generating 30-second OTPs.

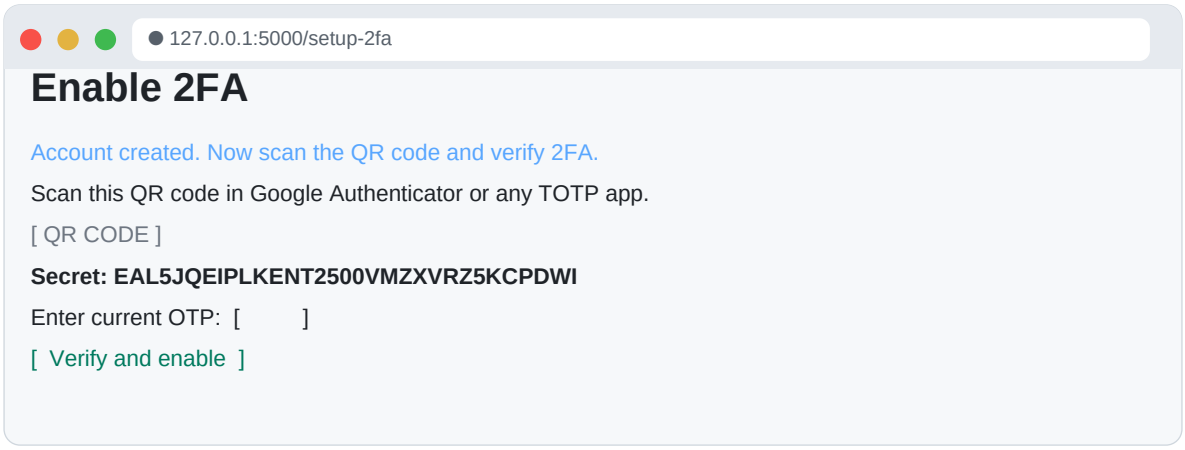

Figure 7 – 2FA setup page for user johann1 showing QR code and TOTP secret

3.2 Successful 2FA Login

After scanning the QR code and entering the current OTP from Google Authenticator, the user is redirected to the dashboard. The session is established only after both password and OTP verification pass.

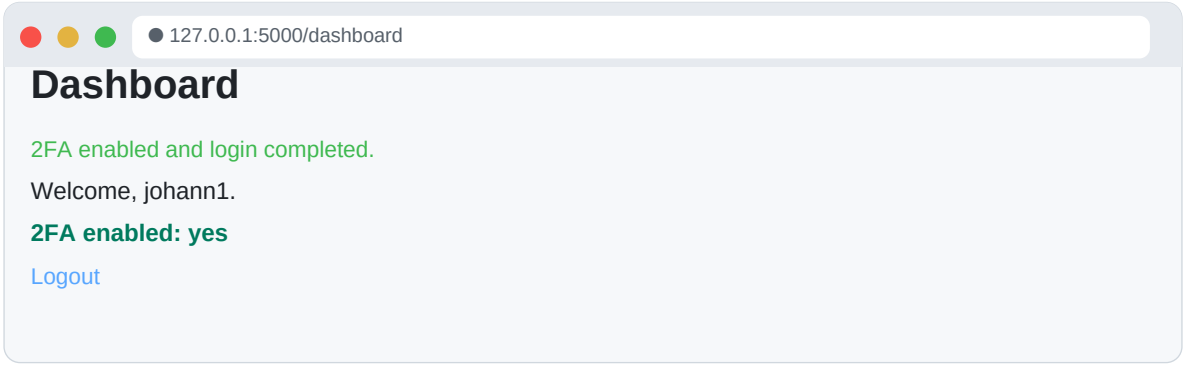

Figure 8 – Dashboard confirming successful 2FA login (2FA enabled: yes)

3.3 Wrong OTP Rejection

If an incorrect OTP is entered during 2FA setup or login, the application rejects it and prompts the user to try a fresh code. The error message is shown below:

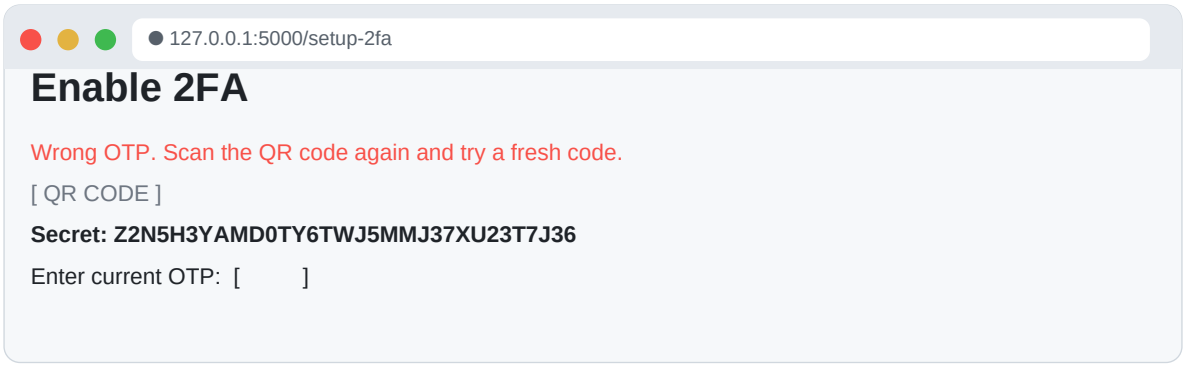

Figure 9 – Wrong OTP rejection message

3.4 QR Code for Second Registered Account

A second account (**johann**) was also registered, demonstrating that each user receives an independent TOTP secret and QR code.

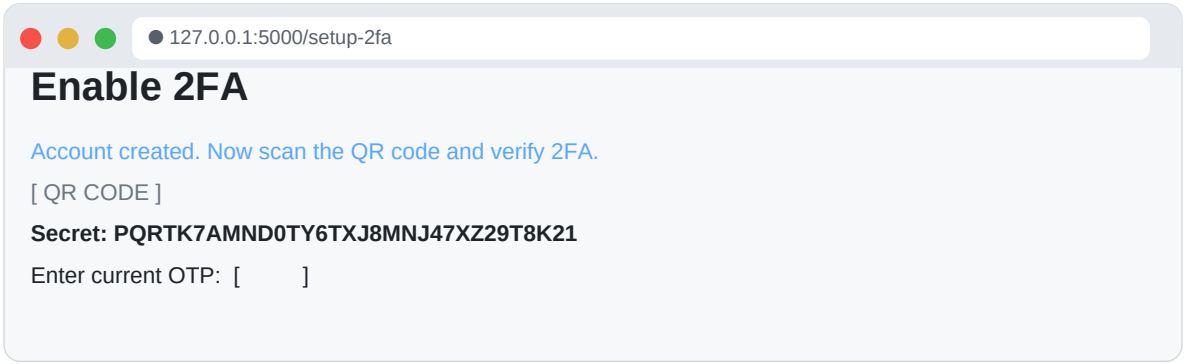

Figure 10 – 2FA setup page for the second account (johann) with a distinct TOTP secret

3.5 Replay Attack Prevention

The application stores `last_totp_counter` in the database. On each 2FA verification, the current 30-second counter is compared with the stored value. If the counter has not advanced (i.e., the same OTP is submitted twice within its validity window), the attempt is rejected with *'Replay rejected. Use a fresh OTP.'*

```
counter = current_counter()  # int(time.time()) // 30
if counter <= user["last_totp_counter"]:
    flash("Replay rejected. Use a fresh OTP.")
    return redirect(url_for("verify_2fa"))
if totp.verify(otp, valid_window=1):
    conn.execute("UPDATE users SET last_totp_counter=? WHERE id=?", (counter, uid))
```

4 Summary and Security Analysis

This lab assignment demonstrates three complementary layers of authentication security:

1. **bcrypt password hashing**
   Passwords are never stored in plaintext. The $2b$12$ prefix confirms bcrypt with cost factor 12, making offline dictionary attacks computationally expensive.

2. **Rate limiting and account lockout**
   Flask-Limiter enforces a 5-requests-per-minute ceiling on /login. After 5 consecutive wrong passwords the account is locked for 15 minutes, preventing brute-force and credential stuffing attacks even from a single IP.

3. **TOTP 2FA with replay protection**
   pyotp generates RFC 6238-compliant time-based OTPs. Each OTP counter is stored after use so that replaying the same code within its 30-second window is explicitly rejected.

Full Source Code → `https://github.com/JohannMartins/SSE-1`